組み込みモジュール設計パターン教科書

Q&A で学ぶ ESP32 モジュール設計の基礎

バージョン	1.0
作成日	2026 年 3 月 14 日
著者	Shiozawa

本書は、ESP32 を題材にした組み込みシステム向け
「モジュール設計パターン」を Q&A 形式で解説する教科書です。

クラスの基礎から 3 フェーズ実行モデルまで、
段階的に学べる構成になっています。

対象読者と読み方

「C++ のクラスはなんとなく知ってるけど、
部品化の経験がない」方を主な対象としています。

第 1 章「前提知識」ではクラスの基礎を解説しています。

構造体・クラス・コンストラクタを既にご存知の方は、
第 1 章を飛ばして第 2 章から読み始めてください。

目次

1 前提知識 — C++ クラスの基礎 2

2 全部 main.cpp に書くとどうなる？ 6
- 2.1 問題 1: 肥大化 . . . 8
- 2.2 問題 2: マジックナンバー . . . 8
- 2.3 問題 3: 再利用できない . . . 8

3 モジュールという考え方 9

4 インターフェースで統一する 12
- 4.1 virtual とは . . . 13
- 4.2 = 0（純粋仮想関数）とは . . . 13
- 4.3 配列で回せるメリット . . . 13

5 データの流れを整理する 15
- 5.1 なぜ必要なの？ . . . 16

6 3 フェーズモデル 18

7 実践 — センサーモジュールを作る 21
- 7.1 ステップ 1: Config 構造体 . . . 21
- 7.2 ステップ 2: Data 構造体 . . . 21
- 7.3 ステップ 3: クラス宣言（.h ファイル） . . . 21
- 7.4 ステップ 4: メソッド実装（.cpp ファイル） . . . 22
- 7.5 ステップ 5: SystemData に追加 . . . 23
- 7.6 ステップ 6: ProjectConfig.h に追加 . . . 23
- 7.7 ステップ 7: main.cpp に追加 . . . 23

8 設計のコツ集 25

付録 A: PlatformIO のススメ 29

1　前提知識 — C++ クラスの基礎

この章では、本書を読むために必要な C++ クラスの基礎知識を解説します。

▷ すでにクラスを理解している方へ

構造体・クラス・コンストラクタ・アクセス修飾子を理解している方は、この章を飛ばして次の「全部 main.cpp に書くとどうなる？」に進んでください。

Q: 構造体（struct）って何？

プログラムを書いていると、「関連する変数をセットで扱いたい」場面がよくあります。
たとえば、LED の状態を管理する変数が 2 つあるとします。

× バラバラの変数で管理

```
bool ledState = false;
uint8_t ledBrightness = 0;

bool led2State = false;
uint8_t led2Brightness = 0;
// LED が増えるたびに変数が倍増...
```

LED が増えるたびに変数名を考えなくてはいけませんし、「どの変数がどの LED のものか」が分かりにくくなります。
そこで「構造体」の出番です。関連する変数を 1 つの型としてまとめられます。

✓ 構造体でまとめる

```
struct LedData {
    bool state = false;
    uint8_t brightness = 0;
};

LedData led1;  // LED1のデーター式
LedData led2;  // LED2のデーター式

led1.state = true;        // ドット(.)でアクセス
led2.brightness = 128;
```

構造体を使えば、「LED に必要なデータは何か」が型として明確になり、LED が何個増えても変数の定義は 1 行で済みます。

Q: 構造体とクラスって何が違うの？

構造体は「データをまとめる」ものでした。クラスはさらに「そのデータを操作する関数」も一緒にまとめます。

✓ クラス ＝ データ ＋ 関数

```
class LedModule {
private:
    uint8_t _pin;        // データ（メンバ変数）
    bool _state;

public:
    // コンストラクタ（後述）
    LedModule(uint8_t pin) : _pin(pin), _state(false) {}

    // メンバ関数
    void turnOn() {
        _state = true;
        digitalWrite(_pin, HIGH);
    }

    void turnOff() {
        _state = false;
        digitalWrite(_pin, LOW);
    }
};
```

構造体でもメンバ関数を持てますが、C++ の慣習として次のように使い分けます。

- **struct** — データの入れ物（Config、Data など）
- **class** — データ＋操作をセットにしたもの（モジュールなど）

▷ struct と class の技術的な違い

C++ では struct と class の唯一の違いは「デフォルトのアクセス修飾子」です。struct はデフォルト public、class はデフォルト private です。本書ではデータの入れ物に struct、操作を持つものに class を使います。

Q: コンストラクタって何？

コンストラクタは「オブジェクトが作られるときに自動で呼ばれる関数」です。初期化忘れを防げます。

× 初期化忘れの危険

```
LedModule led;          // ピン番号が未設定！
led.turnOn();           // どのピンが光る？→バグ
```

✓ コンストラクタで確実に初期化

```
class LedModule {
private:
    uint8_t _pin;
    bool _state;

public:
    // コンストラクタ — 初期化子リストで初期化
    LedModule(uint8_t pin) : _pin(pin), _state(false) {
        // ピンを指定しないと作れない
    }
};

LedModule led(13);   // OK: ピン13で作成
// LedModule led2;   // コンパイルエラー！ピン未指定
```

コロン（:）の後に書く _pin(pin), _state(false) の部分を「初期化子リスト」と呼びます。メンバ変数を効率よく初期化する書き方です。

Q: private/public って何？

クラスの中身を「外から触れる部分」と「触れない部分」に分けるしくみです。

イメージとしては、銀行の窓口を思い浮かべてください。

- **public**（窓口） — お客さん（外部のコード）がアクセスできる
- **private**（金庫） — 銀行員（クラス内部）だけがアクセスできる

```
class LedModule {
private:
    uint8_t _pin;    // 外から直接変更させない

public:
    LedModule(uint8_t pin) : _pin(pin) {}
    void turnOn();   // 外から呼べる
};

LedModule led(13);
led.turnOn();       // OK: publicなので呼べる
// led._pin = 99;   // コンパイルエラー！privateには触れない
```

なぜわざわざ隠すのか？ 外部から _pin を勝手に変更されると、LED モジュールの動作が壊れる可能性があるからです。「触ってほしくないものは private にする」のが安全設計の基本です。

▷ 命名の慣習

private メンバ変数には先頭にアンダースコア（_）を付ける慣習があります。_pin、_state のように書くと、「これは内部変数だな」と一目で分かります。

Q: virtual って何？（予告）

`virtual` は「クラスの継承」という仕組みで使うキーワードです。これを使うと、異なるクラスを同じ型として扱えるようになります。
……と言われてもピンと来ないと思います。
大丈夫です。`virtual` の詳しい説明は本編の「インターフェースで統一する」の章で、具体例を交えて解説します。ここでは「そういうキーワードがあるんだな」とだけ覚えておいてください。

▷ ここまでのまとめ

- **構造体** — 関連する変数をまとめる箱
- **クラス** — データ＋操作をセットにしたもの
- **コンストラクタ** — オブジェクト生成時に自動実行される初期化関数
- **private/public** — アクセス範囲を制限して安全性を確保
- **virtual** — 「インターフェースで統一する」の章で詳しく！

これだけ分かっていれば、本編を読み進められます。さあ、次の章に進みましょう！

2　全部 main.cpp に書くとどうなる？

▷ C++ クラスの復習が必要な方へ

この章から C++ のクラスを使います。構造体・クラス・コンストラクタに自信がある方はそのまま読み進めてください。不安な方は第 1 章「前提知識 — C++ クラスの基礎」（2 ページ）で復習できます。

Q: 全部 main.cpp に書くの、何がまずいの？

まずは「よくある書き方」を見てみましょう。LED2 個と温度センサー 1 個を制御するプログラムです。

× 全部 main.cpp にベタ書き

```cpp
#include <Arduino.h>

// ピン番号（マジックナンバー）
#define LED1_PIN 2
#define LED2_PIN 4
#define TEMP_PIN 34

// グローバル変数がバラバラに並ぶ
bool led1State = false;
bool led2State = false;
float temperature = 0.0;
unsigned long lastBlink = 0;
unsigned long lastRead = 0;

void setup() {
    Serial.begin(115200);
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);
    // 温度センサーはアナログなのでpinMode不要
}

void loop() {
    unsigned long now = millis();

    // LED1: 500msごとに点滅
    if (now - lastBlink >= 500) {
        lastBlink = now;
        led1State = !led1State;
        digitalWrite(LED1_PIN, led1State ? HIGH : LOW);
    }

    // LED2: 常時点灯
    digitalWrite(LED2_PIN, HIGH);

    // 温度センサー: 1秒ごとに読み取り
    if (now - lastRead >= 1000) {
        lastRead = now;
        int raw = analogRead(TEMP_PIN);
        temperature = raw * 3.3 / 4095.0 * 100.0;
        Serial.print("Temp: ");
        Serial.println(temperature);
    }

    // ここにモーター制御を追加したら？
    // さらにディスプレイ表示を追加したら？
    // ...どんどん膨れ上がる
}
```

動くことは動きます。でも、この書き方には 3 つの大きな問題があります。

2.1 問題 1: 肥大化

今は約 40 行ですが、モーター制御、ディスプレイ表示、通信機能……と追加していくと、あっという間に数百行になります。「LED1 の点滅ロジックはどこだっけ？」と探すだけで一苦労です。

2.2 問題 2: マジックナンバー

`LED1_PIN`、`500`（点滅間隔）、`3.3 / 4095.0 * 100.0`（変換式）といった数値がコード中に散らばっています。後から見て「この数字は何？」となりがちです。

2.3 問題 3: 再利用できない

別のプロジェクトでも LED 点滅が必要になったとき、このコードから LED 関連の部分だけを抜き出すのは大変です。変数名が被ったり、依存関係が絡み合っていたりします。

Q: この教科書で何を学ぶの？

この教科書では、上のような「ベタ書きコード」を段階的に改善していきます。

1. **第 3 章: モジュールという考え方** — 1 つのハードウェアを 1 つのクラスに分離する
2. **第 4 章: インターフェースで統一する** — 異なるモジュールを同じ方法で扱う
3. **第 5 章: データの流れを整理する** — モジュール間のデータ受け渡しを安全にする
4. **第 6 章: 3 フェーズモデル** — loop() の処理を「入力→ロジック→出力」に整理する
5. **第 7 章: 実践** — 温度センサーモジュールを一から作る
6. **第 8 章: 設計のコツ集** — 実務で役立つ Tips

最終的に、先ほどのベタ書きコードがどう変わるか？ それは第 6 章で明らかになります。

3　モジュールという考え方

Q: モジュールって何？

モジュールとは、「1 つのハードウェア部品の制御を、1 つのクラスにまとめたもの」です。
ルールはシンプルです。

1 ハードウェア ＝ 1 クラス（1 モジュール）

第 1 章のベタ書きコードから、LED 部分だけを抜き出してクラスにしてみましょう。

✓ LedModule を抽出

```
// LedModule.h
class LedModule {
private:
    uint8_t _pin;
    bool _state;

public:
    LedModule(uint8_t pin) : _pin(pin), _state(false) {}

    bool init() {
        pinMode(_pin, OUTPUT);
        return true;
    }

    void turnOn() {
        _state = true;
        digitalWrite(_pin, HIGH);
    }

    void turnOff() {
        _state = false;
        digitalWrite(_pin, LOW);
    }

    void toggle() {
        _state ? turnOff() : turnOn();
    }
};
```

これで main.cpp は次のようにすっきりします。

```
LedModule led1(2);    // ピン2のLED
LedModule led2(4);    // ピン4のLED

void setup() {
```

```
    led1.init();
    led2.init();
}
```

LED の制御に関するコードは LedModule の中に閉じ込められました。main.cpp を見ても「LED が 2 つあって、init している」ということが一目で分かります。

Q: なんで Config 構造体が必要なの？

先ほどの例では、コンストラクタにピン番号だけ渡しました。でも設定項目が増えるとどうなるでしょう？

× 引数が増えすぎる

```
// ピン番号、初期状態、PWMチャンネル、PWM周波数...
LedModule led(2, false, 0, 5000);
// どの引数が何を意味するか分からない！
```

✓ Config 構造体にまとめる

```
// 設定をまとめた構造体
struct LedConfig {
    uint8_t pin;
};

class LedModule {
private:
    LedConfig _config;  // 設定を丸ごと保持

public:
    LedModule(const LedConfig& config) : _config(config) {}

    bool init() {
        pinMode(_config.pin, OUTPUT);
        return true;
    }
};

// 使う側 — 何の設定か一目瞭然
const LedConfig LED1_CONFIG = { .pin = 2 };
LedModule led1(LED1_CONFIG);
```

Config 構造体を使うメリットは 3 つです。

1. 設定項目が増えても、構造体にフィールドを追加するだけ
2. 設定値を 1 か所（ProjectConfig.h）にまとめて管理できる
3. コンストラクタの引数の意味が明確

Q: Config と Data の違いは？

モジュールが扱う情報は、大きく 2 種類に分かれます。

	Config	Data
意味	起動時の設定	実行中に変化するデータ
いつ決まる？	プログラム開始前（コンパイル時）	プログラム実行中
変わる？	基本的に変わらない	常に変わる
例	ピン番号、I2C アドレス	LED 状態、センサー値

```
// Config — 起動時に決まる設定
struct LedConfig {
    uint8_t pin;        // ピン番号（変わらない）
};

// Data — 実行中に変わるデータ
struct LedData {
    bool state = false;         // ON/OFF状態
    uint8_t brightness = 0;     // 明るさ
};
```

▷ なぜ分けるの？

Config と Data を分けることで、「この値は実行中に変わるのか？」が型レベルで明確になります。Config は `const` にできるので、うっかり書き換えてしまうバグも防げます。

4　インターフェースで統一する

Q: モジュールが増えたらどうするの？

LED、温度センサー、モーター……とモジュールが増えていくと、こんなコードになりがちです。

× 呼び方がバラバラ

```
void setup() {
    led.init();
    tempSensor.begin();        // beginだったり
    motor.start();             // startだったり
    display.initialize();      // initializeだったり
}

void loop() {
    led.toggle();
    tempSensor.readValue();    // メソッド名もバラバラ
    motor.drive();
    display.refresh();
}
```

モジュールごとに初期化メソッドの名前が違う、更新メソッドの名前が違う。これでは新しいモジュールを追加するたびに「このモジュールは何を呼べばいいんだっけ？」と調べる羽目になります。
もっと困るのは、「全モジュールの初期化を順番に実行する」ような共通処理が書けないことです。

Q: IModule って何をしてくれるの？

IModule は「すべてのモジュールが守るべきルール」を定めたインターフェースです。

✓ IModule インターフェース

```
struct SystemData;  // 前方宣言（第4章で詳しく説明）

class IModule {
public:
    bool enabled = true;

    virtual bool init() = 0;
    virtual void update(SystemData& data) = 0;
    virtual void deinit() {}   // デフォルト空実装
    virtual ~IModule() {}
};
```

ここで出てくる virtual と = 0 について説明しましょう。

4.1 virtual とは

virtual は「この関数は子クラスで上書き（オーバーライド）できますよ」という印です。

4.2 = 0（純粋仮想関数）とは

= 0 は「この関数は子クラスで必ず実装してね。実装しないとコンパイルエラーにするよ」という意味です。つまり、IModule を継承するクラスは init() と update() を必ず書かなくてはいけません。

4.3 配列で回せるメリット

すべてのモジュールが IModule を継承すると、こんなことができます。

✓ 配列でまとめて初期化

```
// 異なるクラスでも、IModule*の配列に入れられる
IModule* modules[] = {
    &led,
    &tempSensor,
    &motor,
};

// まとめて初期化
for (int i = 0; i < 3; i++) {
    if (!modules[i]->init()) {
        Serial.println("init failed!");
        modules[i]->enabled = false;
    }
}

// まとめて更新
void loop() {
    for (int i = 0; i < 3; i++) {
        if (modules[i]->enabled) {
            modules[i]->update(systemData);
        }
    }
}
```

LedModule も TempSensorModule も MotorModule も、すべて IModule*として統一的に扱えます。新しいモジュールを追加しても、for 文はそのままです。

▷ override キーワード

子クラスで親の仮想関数を実装するときは、override キーワードを付けましょう。タイプミスで別の関数を定義してしまうバグを防げます。

```
class LedModule : public IModule {
public:
    bool init() override { /* ... */ }       // OK
    void update(SystemData& data) override; // OK
};
```

Q: なんで enabled があるの？

ハードウェアは壊れることがあります。センサーが接触不良で応答しない、ディスプレイの初期化に失敗した……そんなとき、プログラム全体を止めたくはありません。

enabled フラグを使えば、初期化に失敗したモジュールだけをスキップできます。

```
for (int i = 0; i < MODULE_COUNT; i++) {
    if (!modules[i]->init()) {
        Serial.println("init failed, disabling");
        modules[i]->enabled = false;  // このモジュールだけ無効化
    }
}

void loop() {
    for (int i = 0; i < MODULE_COUNT; i++) {
        if (modules[i]->enabled) {    // 無効なモジュールはスキップ
            modules[i]->update(systemData);
        }
    }
}
```

温度センサーが壊れても、LED やモーターは動き続けます。組み込みシステムでは「部分的に動く」ことが重要です。

5　データの流れを整理する

Q: モジュール同士が直接話すとなぜダメ？

「温度が 30 度を超えたら LED を点灯」したいとき、こんなコードを書きたくなるかもしれません。

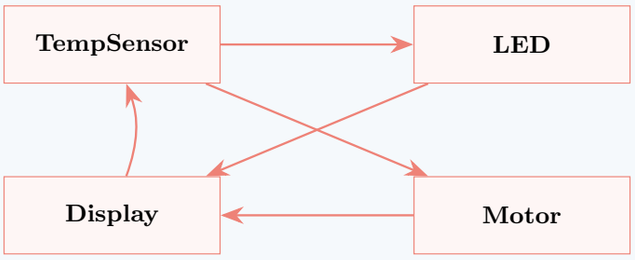
× モジュール同士が直接参照

```
class TempSensorModule {
private:
    LedModule* _led;  // LEDモジュールを直接持っている！

public:
    void update() {
        float temp = readSensor();
        if (temp > 30.0) {
            _led->turnOn();  // 直接操作
        }
    }
};
```

一見うまく動きそうですが、以下の問題が起きます。

- 温度センサーモジュールが LED モジュールに依存 → LED なしでは使えない
- 「温度でモーターも回したい」→ MotorModule*も追加？ → 依存がどんどん増える
- モジュール間の依存が複雑に絡み合い、変更が困難に（スパゲッティ化）



矢印（依存関係）が蜘蛛の巣のように広がっています。これでは 1 つのモジュールを変更すると、他のモジュールにも影響が波及します。

Q: SystemData って何？

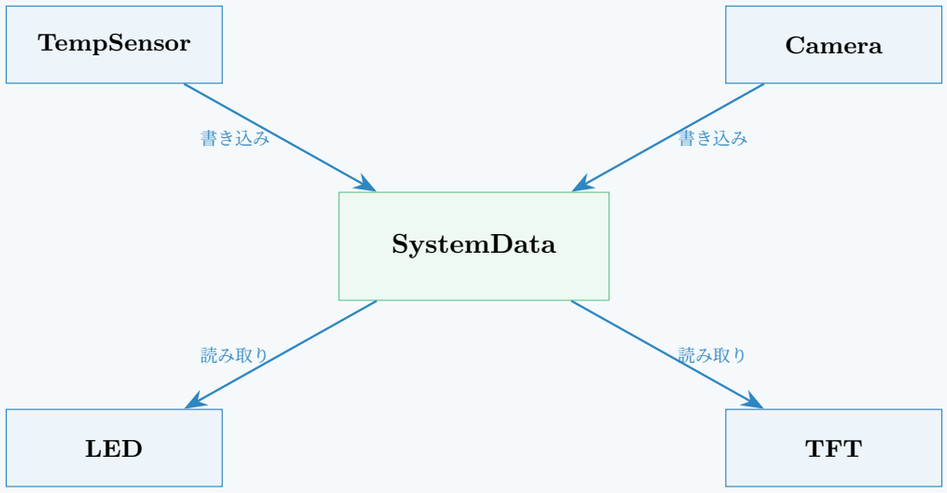
`SystemData` は、すべてのモジュールのデータを 1 か所にまとめた「伝言板」です。

✓ SystemData — モジュールの伝言板

```
#include "LedModule.h"
#include "TempSensorModule.h"

struct SystemData {
    LedData led;              // LEDの状態
    TempSensorData temp;      // 温度センサーの値
};
```

モジュール同士は直接会話しません。代わりに SystemData を介してデータをやり取りします。



- 入力モジュール（TempSensor 等）は SystemData にデータを書き込む
- 出力モジュール（LED 等）は SystemData からデータを読み取る
- モジュール同士は直接参照しない → 依存関係がシンプル

温度センサーは LED の存在を知りません。ただ SystemData の `temp` に値を書くだけです。LED をモーターに差し替えても、温度センサーのコードは 1 行も変わりません。

Q: 前方宣言って何？

IModule.h の中にこんな 1 行がありました。

```
struct SystemData;  // 前方宣言
```

これは「SystemData という型がどこかで定義されてますよ」とコンパイラに教える宣言です。

5.1 なぜ必要なの？

IModule.h は `update(SystemData& data)` を宣言していますが、SystemData の中身を知る必要はありません（ポインタや参照で受け取るだけなので）。
もし IModule.h が `SystemData.h` をインクルードすると、こんな問題が起きます。

× 循環依存

```
// IModule.h が SystemData.h をインクルード
// SystemData.h が LedModule.h をインクルード
// LedModule.h が IModule.h をインクルード
// → IModule.h が SystemData.h をインクルード → 無限ループ！
```

前方宣言を使えば、この循環を断ち切れます。

✓ 前方宣言で循環を回避

```
// IModule.h — SystemData.hをインクルードしない
struct SystemData;  // 「こういう型がある」とだけ宣言

class IModule {
public:
    virtual void update(SystemData& data) = 0;
    // 参照で受け取るだけなので、中身を知らなくてOK
};
```

▷ ヘッダーと cpp の使い分け

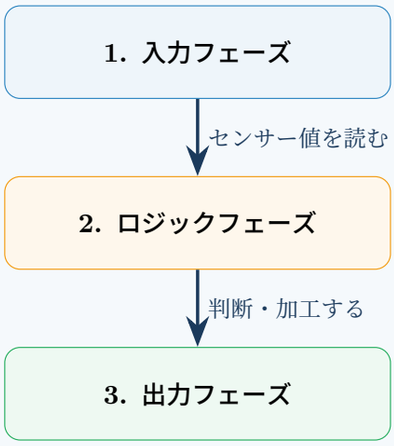
- **.h ファイル** — `struct SystemData;` の前方宣言のみ
- **.cpp ファイル** — `#include "SystemData.h"` でフル定義を使う

.cpp では実際に SystemData のメンバにアクセスするので、フル定義が必要です。このルールを守れば循環依存は起きません。

6　3 フェーズモデル

Q: loop() の中身はどう整理するの？

ここまでで、モジュール・インターフェース・SystemData の 3 つの道具が揃いました。最後のピースは「loop() の実行順序」です。
loop() の中身を 3 つのフェーズに分けます。これを「3 フェーズモデル」と呼びます。



工場の流れ作業に例えると分かりやすいです。

1. **入力フェーズ**（検品係） — センサーからデータを読み取り、SystemData に書き込む
2. **ロジックフェーズ**（判断係） — SystemData を見て、どう動くか判断する
3. **出力フェーズ**（作業係） — SystemData の指示に従い、LED やモーターを制御する

✓ 3 フェーズの loop()

```
// モジュール配列
IModule* inputModules[] = { &tempSensor };
IModule* outputModules[] = { &led };
const int INPUT_COUNT = 1;
const int OUTPUT_COUNT = 1;

void loop() {
    // 1. 入力フェーズ — センサー値をSystemDataに書き込む
    for (int i = 0; i < INPUT_COUNT; i++) {
        if (inputModules[i]->enabled) {
            inputModules[i]->update(systemData);
        }
    }

    // 2. ロジックフェーズ — データを見て判断する
    applyLogic(systemData);

    // 3. 出力フェーズ — 判断結果に従い制御する
    for (int i = 0; i < OUTPUT_COUNT; i++) {
        if (outputModules[i]->enabled) {
            outputModules[i]->update(systemData);
        }
    }
}
```

Q: なんで順番が大事なの？

「入力→ロジック→出力」の順番を守らないと、思わぬバグが発生します。

× 出力が先に実行される場合

```
void loop() {
    // 出力が先！
    led.update(systemData);    // 前回のデータで光ってしまう

    // 入力が後
    tempSensor.update(systemData);  // 今回のデータを読む

    applyLogic(systemData);  // 今回のデータで判断するが...
    // → LEDにはもう反映されない（次のloopまで待つ）
}
```

出力が先だと、LED は常に「1 ループ前のデータ」で動作します。温度が急上昇しても、LED の反応が 1 ループ分遅れます。通常は気にならない程度ですが、リアルタイム性が求められるシステムでは致命的です。また、ロジックフェーズをスキップして入力モジュールが出力モジュールのデータを直接書き換えると、「なぜこの LED が光っているのか」の理由がコード上で追えなくなります。

！ クロス参照の禁止

入力モジュールが出力モジュールのデータを読み書きしたり、その逆をしたりすることを「クロス参照」と呼びます。これは禁止です。

- 入力モジュール → 自分の Data に書き込むだけ
- 出力モジュール → 自分の Data を読み取るだけ
- データの受け渡し → ロジックフェーズが担当

Q: ロジックフェーズでは何を書くの？

ロジックフェーズは「入力データを見て、出力データを決める」場所です。

✓ applyLogic() の例

```
void applyLogic(SystemData& data) {
    // 温度が30度を超えたらLEDを点灯
    if (data.temp.temperature > 30.0) {
        data.led.state = true;
    } else {
        data.led.state = false;
    }
}
```

ポイントは以下の通りです。

- ハードウェアを直接操作しない（`digitalWrite` 等を呼ばない）
- SystemData の値を読んで、SystemData の値を書き換えるだけ
- 「なぜ LED が光るのか」のルールがこの関数に集約される

ロジックが複雑になっても、この関数を読めば動作の全体像が分かります。

▷ 第 1 章のコードがこう変わった

第 2 章では 60 行のベタ書きでしたが、3 フェーズモデルで整理すると次のような構成になります。

- `LedModule` — LED の制御（約 20 行）
- `TempSensorModule` — 温度の読み取り（約 20 行）
- `applyLogic()` — ルール（約 5 行）
- `main.cpp` — 配列定義 + 3 フェーズループ（約 30 行）

合計行数は増えますが、各ファイルが短く、役割が明確です。

7　実践 — センサーモジュールを作る

この章では、温度センサーモジュールを一から作ります。第 3 章〜第 6 章で学んだ知識の総まとめです。

Q: 新しいモジュールはどう作るの？

新しいモジュールを作る手順は、いつも同じ 7 ステップです。

1. Config 構造体を定義する
2. Data 構造体を定義する
3. クラスを宣言する（.h ファイル）
4. メソッドを実装する（.cpp ファイル）
5. SystemData に Data を追加する
6. ProjectConfig.h に Config インスタンスを追加する
7. main.cpp のモジュール配列に追加する

順番に見ていきましょう。analogRead() で温度を読む簡易センサーを題材にします。

7.1　ステップ 1: Config 構造体

起動時に決まる設定をまとめます。

```
// TempSensorModule.h 内で定義
struct TempSensorConfig {
    uint8_t analogPin;          // アナログ入力ピン
    float conversionFactor;     // ADC値 → 温度の変換係数
};
```

7.2　ステップ 2: Data 構造体

実行中に変化するデータをまとめます。デフォルト値を必ず設定します。

```
// TempSensorModule.h 内で定義
struct TempSensorData {
    float temperature = -999.0f;  // 温度（未取得時は異常値）
    bool isValid = false;         // データが有効か
};
```

▷ なぜデフォルト値を設定するの？

init() 前やセンサー故障時でも、SystemData の値は参照される可能性があります。デフォルト値がないと、不定値（ゴミデータ）が入ったまま使われてしまいます。`-999.0f` のような明らかにおかしい値を入れておけば、「まだ読めていない」ことが一目で分かります。

7.3　ステップ 3: クラス宣言（.h ファイル）

```
// TempSensorModule.h
```

```
#pragma once
#include <Arduino.h>
#include "IModule.h"
#include "ModuleTimer.h"

struct TempSensorConfig { /* (上で定義済み) */ };
struct TempSensorData   { /* (上で定義済み) */ };

struct SystemData;  // 前方宣言

class TempSensorModule : public IModule {
private:
    TempSensorConfig _config;
    ModuleTimer _readTimer;

public:
    TempSensorModule(const TempSensorConfig& config);
    bool init() override;
    void update(SystemData& data) override;
};
```

ポイント:

- `#pragma once` — 二重インクルード防止
- `struct SystemData;` — 前方宣言（SystemData.h はインクルードしない）
- `: public IModule` — IModule を継承
- `override` — 親クラスの仮想関数を上書きすることを明示

7.4 ステップ 4: メソッド実装（.cpp ファイル）

```
// TempSensorModule.cpp
#include "TempSensorModule.h"
#include "SystemData.h"  // ここでフル定義をインクルード

TempSensorModule::TempSensorModule(const TempSensorConfig& config)
    : _config(config) {}

bool TempSensorModule::init() {
    // アナログピンはpinMode不要 (ESP32)
    _readTimer.setTime();
    Serial.println("[TempSensor] init OK");
    return true;
}

void TempSensorModule::update(SystemData& data) {
    // 1秒ごとに温度を読み取る
    if (_readTimer.getNowTime() < 1000) return;
    _readTimer.setTime();

    int raw = analogRead(_config.analogPin);
    data.temp.temperature = raw * _config.conversionFactor;
```

```
    data.temp.isValid = true;
}
```

7.5 ステップ 5: SystemData に追加

```
// SystemData.h
#include "LedModule.h"
#include "TempSensorModule.h"

struct SystemData {
    LedData led;
    TempSensorData temp;   // 追加！
};
```

7.6 ステップ 6: ProjectConfig.h に追加

```
// ProjectConfig.h
#include "TempSensorModule.h"

const TempSensorConfig TEMP_CONFIG = {
    .analogPin = 34,
    .conversionFactor = 3.3f / 4095.0f * 100.0f,
};
```

7.7 ステップ 7: main.cpp に追加

```
// main.cpp
TempSensorModule tempSensor(TEMP_CONFIG);

IModule* inputModules[] = {
    &tempSensor,   // 入力モジュール配列に追加
};
```

以上で完成です。新しいモジュールを追加するときも、同じ 7 ステップを繰り返すだけです。

Q: ModuleTimer はどう使うの？

組み込みプログラムでありがちなのが、delay() を使った待ち時間の実装です。

× delay() で待つ

```
void update(SystemData& data) {
    int raw = analogRead(_config.analogPin);
    data.temp.temperature = raw * _config.conversionFactor;
    delay(1000);  // ここで1秒間、何もできない！
}
```

delay(1000) を呼ぶと、プログラム全体が 1 秒間停止します。その間、他のモジュールも動けません。LED の点滅もディスプレイの更新もすべて止まります。

✓ ModuleTimer で周期実行

```
class TempSensorModule : public IModule {
private:
    ModuleTimer _readTimer;  // タイマーをメンバ変数に

public:
    bool init() override {
        _readTimer.setTime();  // 現在時刻を記録
        return true;
    }

    void update(SystemData& data) override {
        // 前回から1秒経っていなければ即リターン
        if (_readTimer.getNowTime() < 1000) return;
        _readTimer.setTime();  // 時刻をリセット

        // 1秒経ったときだけ実行
        int raw = analogRead(_config.analogPin);
        data.temp.temperature = raw * _config.conversionFactor;
        data.temp.isValid = true;
    }
};
```

ModuleTimer は millis() をラップした軽量なタイマーです。

- setTime() — 現在時刻を記録する
- getNowTime() — 前回の setTime() からの経過ミリ秒を返す

delay() と違い、プログラムをブロックしません。経過時間が足りなければ即座に return し、他のモジュールに処理を譲ります。

! delay() は原則禁止

3 フェーズモデルでは、各モジュールの update() はできるだけ速く返る必要があります。delay() を使うと他のすべてのモジュールが影響を受けるため、ModuleTimer を使ったノンブロッキング方式を徹底してください。

8　設計のコツ集

この章では、実装でよくある疑問に短い Q&A 形式で答えます。

Q: エラーが起きたらどうする？

init() は bool を返します。失敗したら false を返しましょう。

```
bool TempSensorModule::init() {
    if (!checkSensor()) {
        Serial.println("[TempSensor] init failed: sensor not found");
        return false;  // 失敗を通知
    }
    return true;
}
```

update() でエラーが起きた場合は、Data のフラグで通知します。

```
void TempSensorModule::update(SystemData& data) {
    int raw = analogRead(_config.analogPin);
    if (raw < 0) {
        data.temp.isValid = false;  // エラーフラグ
        return;
    }
    data.temp.temperature = raw * _config.conversionFactor;
    data.temp.isValid = true;
}
```

Q: SPI や I2C を複数モジュールで共有するには？

バス（SPI、I2C）のインスタンスは main.cpp で作り、コンストラクタの引数でモジュールに渡します。

```
// main.cpp
static TwoWire i2cBus = TwoWire(0);

void setup() {
    i2cBus.begin(SDA_PIN, SCL_PIN);  // バス初期化は先に！

    // モジュール初期化は後
    tempSensor.init();
    display.init();
}
```

```
// モジュール側 — ポインタで受け取る
class TempSensorModule : public IModule {
private:
    TwoWire* _wire;

public:
    TempSensorModule(const TempSensorConfig& config,
```

```
                        TwoWire* wire)
        : _config(config), _wire(wire) {}
};
```

！バス初期化の順番

bus.begin() は必ず setup() 内で、全モジュールの init() より前に呼んでください。モジュールの init() 内で bus.begin() を呼ぶと、2 つ目のモジュールが初期化するときにバスがリセットされる可能性があります。

SPI 使用時は排他制御を忘れないでください。

```
void update(SystemData& data) {
    _spi->beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));
    // SPI通信
    _spi->endTransaction();
}
```

Q: init 失敗したモジュールは復帰できる？

できます。ロジックフェーズで定期的に init() を再試行します。

```
// ロジックフェーズ内
void applyLogic(SystemData& data) {
    // 無効化されたモジュールの再init（10秒ごと）
    static ModuleTimer retryTimer;
    if (retryTimer.getNowTime() >= 10000) {
        retryTimer.setTime();
        for (int i = 0; i < INPUT_COUNT; i++) {
            if (!inputModules[i]->enabled) {
                if (inputModules[i]->init()) {
                    inputModules[i]->enabled = true;
                    Serial.println("[System] module recovered");
                }
            }
        }
    }
}
```

接触不良で一時的に通信が切れたセンサーが、再接続後に自動復帰します。

Q: ログはどう書くのがいい？

統一フォーマットを使いましょう。

```
// フォーマット：[モジュール名] メッセージ
Serial.println("[TempSensor] init OK");
Serial.println("[TempSensor] read failed: timeout");
Serial.println("[LED] state changed: ON");
```

角括弧でモジュール名を付けると、シリアルモニターでどのモジュールの出力か一目で分かります。

Q: 実行中に Config を変更したいときは？

Config 変更はロジックフェーズでのみ行います。

× 入力フェーズで Config 変更

```
// BLEモジュールのupdate()内で直接変更 → 危険！
void BleModule::update(SystemData& data) {
    if (newConfigReceived) {
        ledModule._config.pin = newPin;  // 他モジュールを直接操作
    }
}
```

✓ SystemData 経由でロジックフェーズに渡す

```
// BLEモジュール — 受信データをSystemDataに書くだけ
void BleModule::update(SystemData& data) {
    if (newConfigReceived) {
        data.ble.requestedLedPin = newPin;
        data.ble.configUpdateRequested = true;
    }
}

// ロジックフェーズ — ここでConfig変更を実行
void applyLogic(SystemData& data) {
    if (data.ble.configUpdateRequested) {
        // 安全にConfig変更を適用
        data.ble.configUpdateRequested = false;
    }
}
```

Q: deinit() はいつ使うの？

deinit() は、モジュールが確保したリソースを解放するためのメソッドです。IModule にデフォルトの空実装があるので、必要なモジュールだけオーバーライドします。

```
class CameraModule : public IModule {
public:
    void deinit() override {
        // フレームバッファを解放
        if (_fb) {
            esp_camera_fb_return(_fb);
            _fb = nullptr;
        }
        esp_camera_deinit();
        Serial.println("[Camera] deinit done");
    }
};
```

主な用途は以下の通りです。

- ディープスリープに入る前のリソース解放
- モジュールの動的な停止・再起動
- テスト時の後片付け

付録 A: PlatformIO のススメ

この付録では、本書のサンプルプロジェクトで使用している開発環境「PlatformIO」について解説します。

Q: PlatformIO って何？

PlatformIO は、マイコンのプログラムを書いてボードに書き込むための開発環境です。Arduino IDE と同じ役割を果たしますが、大きな違いが 1 つあります。

- **Arduino IDE** — マイコン開発専用のスタンドアロンアプリ
- **PlatformIO** — VSCode（Visual Studio Code）の拡張機能として動作する

Arduino 互換なので、Arduino IDE で書いたコードはほぼそのまま PlatformIO でも動きます。「Arduino IDE に慣れているからコードが書けない」という心配は不要です。

▷ VSCode とは

VSCode（Visual Studio Code）は Microsoft が無料で提供する高機能テキストエディタです。拡張機能を追加することでさまざまな言語・用途に対応でき、現在最も広く使われている開発エディタの一つです。

Q: Arduino IDE と何が違うの？

主な違いを比較表にまとめます。

項目	Arduino IDE	PlatformIO
エディタ	専用エディタ（機能が限定的）	VSCode（補完・定義ジャンプ等が強力）
ライブラリ管理	グローバルに 1 バージョンのみ	プロジェクトごとにバージョン固定
ボード設定	GUI で毎回選択	`platformio.ini` に記述（固定）
環境の共有	口頭やメモで伝える	`platformio.ini` を Git で共有
CI/CD	対応困難	CLI ベースで容易
複数ボード対応	手動で切り替え	`[env:名前]` で同時管理

特に「ライブラリのバージョン管理」と「環境の共有」が PlatformIO の大きな強みです。

Q: チーム開発で PlatformIO が強い理由は？

Arduino IDE でチーム開発をしようとすると、よくこんな問題が起きます。

× Arduino IDE でのチーム開発あるある

- 「ボードマネージャ v2.0.11 じゃないとビルド通らないんだけど…」
- 「ライブラリのバージョン合わせて」→ 口頭伝達 → 忘れる → ビルドが通らない
- 「え、自分の PC では動くんだけどな」

PlatformIO なら、プロジェクトに必要な情報がすべて platformio.ini に記述されています。

✓ PlatformIO なら git clone するだけで同じ環境が再現される

```
git clone https://github.com/yourname/yourproject
cd yourproject
pio run          # 同じplatformio.iniからビルド → 全員同じ結果
```

▷ AI とのデバッグでも強い

ChatGPT や Claude などの AI ツールにデバッグを相談するとき、platformio.ini を貼り付けるだけでプラットフォーム・ボード・ライブラリバージョンが確定した状態で相談できます。「環境が違うのでは？」という的外れな回答が減ります。

Q: platformio.ini の読み方

platformio.ini は、プロジェクトのビルド設定をすべて記述するファイルです。本リポジトリの設定を例に各項目を解説します。

```
[env:esp32-s3-cam-n16r8]    ; ビルド対象の環境定義（名前は自由）
platform = espressif32       ; 使用するマイコンプラットフォーム
board = esp32-s3-devkitc-1   ; 対象ボード
framework = arduino          ; Arduinoフレームワークを使用
monitor_speed = 115200       ; シリアルモニターのボーレート

lib_deps =
    bodmer/TFT_eSPI @ ^2.5.43   ; 使用ライブラリとバージョン

build_flags =
    -D BOARD_HAS_PSRAM      ; コンパイルマクロの定義
    -I include              ; include/ フォルダをパスに追加
```

各項目の役割をまとめます。

項目	説明
[env: 名前]	ビルド対象の環境定義。複数書けば複数ボードを管理できる
platform	使用するマイコンプラットフォーム（例: espressif32）
board	対象ボードの識別子
framework	arduino または espidf 等
lib_deps	使用ライブラリとバージョン。@^2.5.43 はマイナーバージョン以上を許容
build_flags	コンパイルオプション（マクロ定義・インクルードパス等）
monitor_speed	シリアルモニターのボーレート

▷ Arduino IDE との対応

Arduino IDE で言うところの「ボードマネージャの URL 追加 → ボード選択 → ライブラリマネージャでインストール」を、この platformio.ini 1 ファイルで全部済ませています。初回 pio run 時に必要なものが自動でダウンロードされます。

Q: PlatformIO の始め方

以下のステップで環境を構築できます。

1. **VSCode をインストール**
   https://code.visualstudio.com/ からダウンロードしてインストールする
2. **拡張機能「PlatformIO IDE」をインストール**
   VSCode の拡張機能タブ（`Ctrl+Shift+X`）で「PlatformIO IDE」を検索してインストールする
3. **プロジェクトを開く**
   PlatformIO のホーム画面から「Open Project」で既存プロジェクトを開く、
   または「New Project」でボードを選んで新規作成する
4. **ビルドする**
   画面下のチェックマーク（✓）をクリック、またはターミナルで `pio run`
5. **書き込む**
   画面下の矢印（→）をクリック、またはターミナルで `pio run -t upload`
6. **シリアルモニターを開く**
   画面下のコンセントマークをクリック、またはターミナルで `pio device monitor`

▷ 本書のサンプルプロジェクトについて

本書のサンプルプロジェクトも PlatformIO 形式です。`git clone` して VSCode で開くだけで、すぐにビルド・書き込みができます。ライブラリのインストール等は `platformio.ini` が自動で処理します。